

verl: an Open-Source Large-Scale LLM RL Framework for Agentic Tasks

Jiarui Fang
ByteDance

SFT vs RL

Supervised fine-tuning

- Learning from labeled examples
- Play with a single model

Reinforcement learning

- Optimization based on rewards
- Play with multiple models

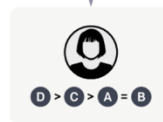
Step 2

Collect comparison data, and train a reward model.

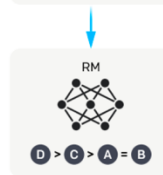
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



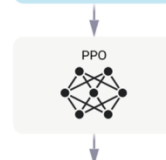
Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.

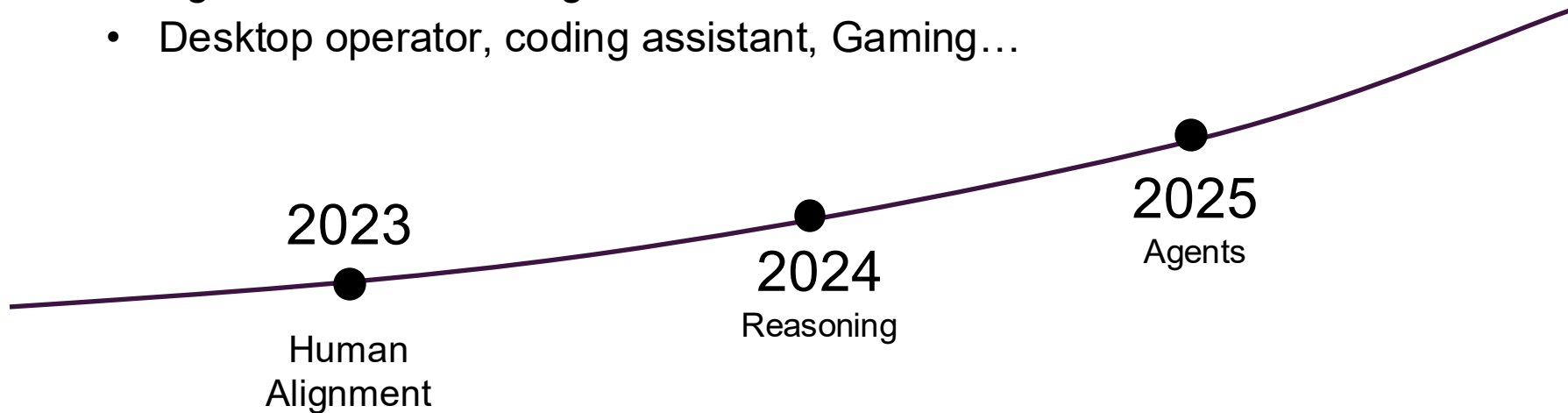


The reward is used to update the policy using PPO.

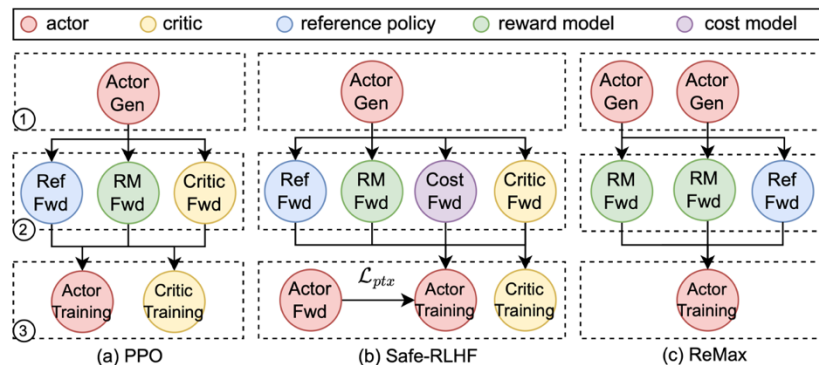


RL is becoming important

- Reinforcement learning (RL) at Bytedance
 - Human Alignment with RLHF
 - Reasoning: O1/Claude-3.7 performance on math benchmarks
 - Image/video/music generation
 - Agentic LLM tool using
 - Desktop operator, coding assistant, Gaming...



RL is Complex Dataflow



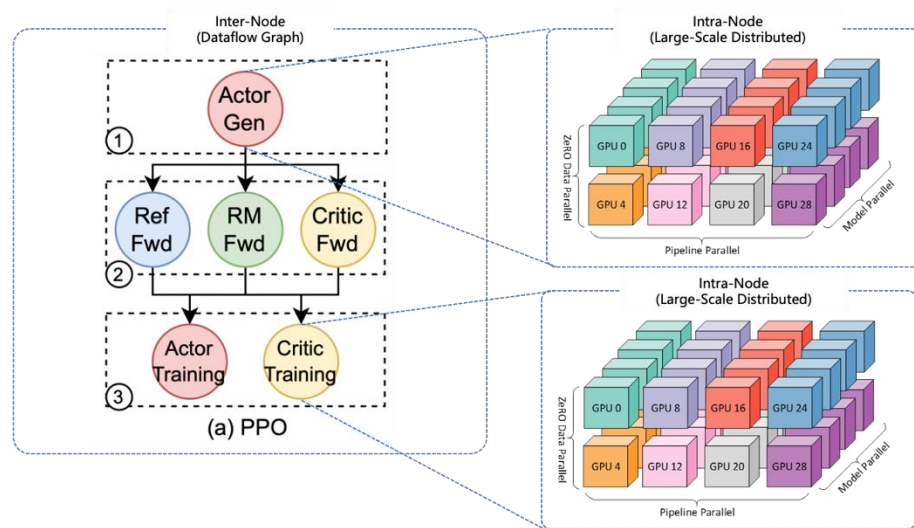
Reinforcement Learning (RL) can be modelled as **complex dataflow graph** ([Schaarschmidt et al. 2019](#); [Liang et al. 2021](#); [Sheng et al. 2025](#)), consisting of:

- **multiple models**: actor, critic, reference, reward model, etc.
- **multiple stages**: generating, preparing experiences, training
- **multiple workloads**: generation, inference, training

RL with LLMs is Large-Scale Distributed Dataflow

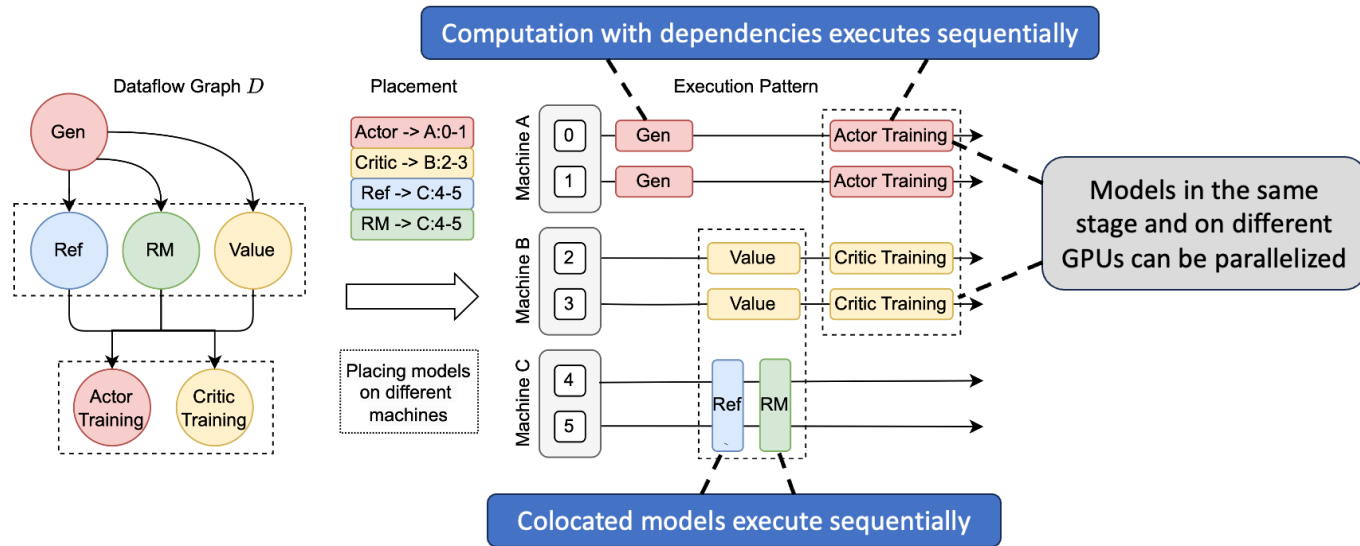
Training needs ND parallelisms (Megatron-LM)

- Growing model size:
Qwen 235B, Deepseek 671B
- Growing sequence length:
8k -> 1M



Each **operator** in the RL dataflow = a large-scale **distributed** computing workload

RL dataflow is hard to implementation: Data Dependencies & Resource Limitations



Implementing RL algorithm with LLMs requires complex trade-offs between various constraints.

Verl's Open-Source Community

<https://github.com/volcengine/verl>

So far, verl has gained:

- 10k+ stars
- 1k+ forks
- 1.1k+ PRs
- 250+ contributors

Many popular RL projects are built on top of verl, including:

- TinyZero (12k stars)
- SimpleRL-Zoo (3.6k stars)
- rllm (3.3k stars)
- SkyThought (3.3k stars)
- OpenManus-RL (2.9k stars)
- ...

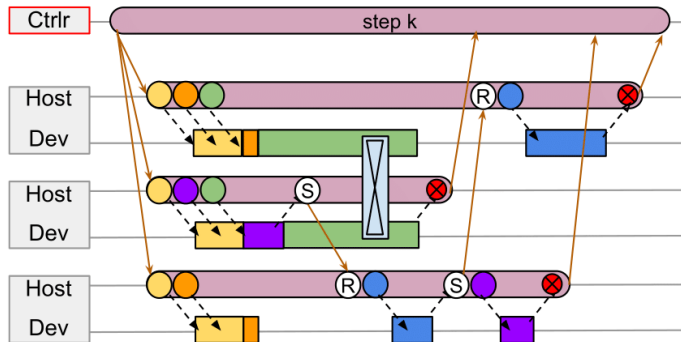
Verl's Highlight Features

- **Easy extension of diverse RL algorithms:** The **hybrid-controller** programming model enables building RL dataflows such as GRPO, PPO in a few lines of code.
- **Efficient actor model resharding with 3D-HybridEngine:** Eliminates memory redundancy and significantly reduces communication overhead during transitions between training and generation phases.
- **Seamless integration of existing LLM infra with modular APIs:** Decouples computation and data dependencies, reuses existing LLM frameworks, such as FSDP, Megatron-LM, vLLM, SGLang, etc
- **Flexible device mapping:** Supports various placement of models onto different sets of GPUs for efficient resource utilization and scalability across different cluster sizes.
- **Wide Model Supports:** besides from Qwen 7/32B small models, large MoEs like DeepSeek 671B/Qwen 235B are also supported.
- **State-of-the-art throughput:** SOTA LLM training and inference engine integrations and SOTA RL throughput.

Verl's Distributed Paradigm: Hybrid-Controller

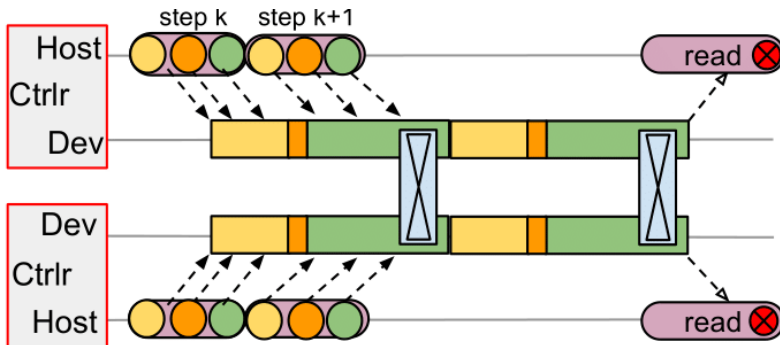
Single-Controller (MPMD):

A centralized controller manages all the workers, running different programs

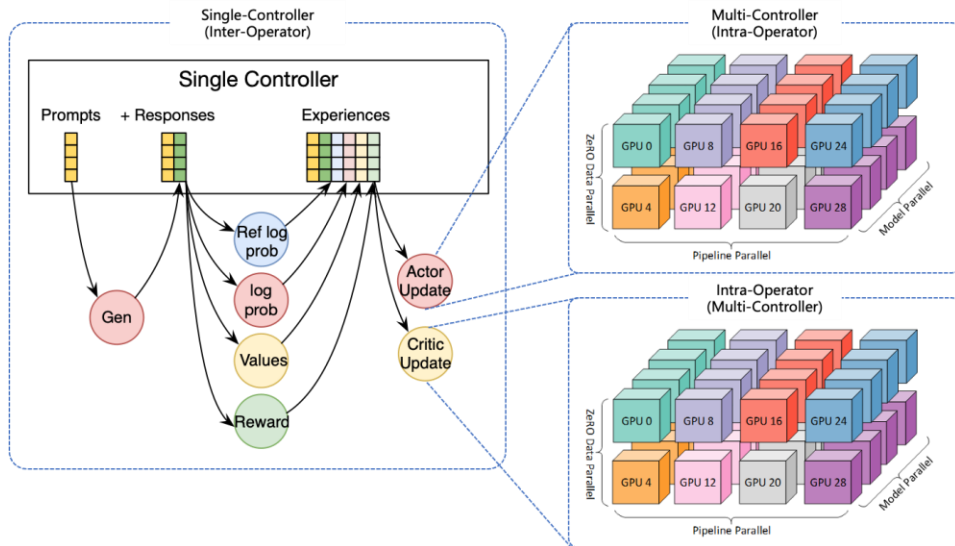


Multi-Controller (SPMD):

Each worker has its own controller, running the same program with different data

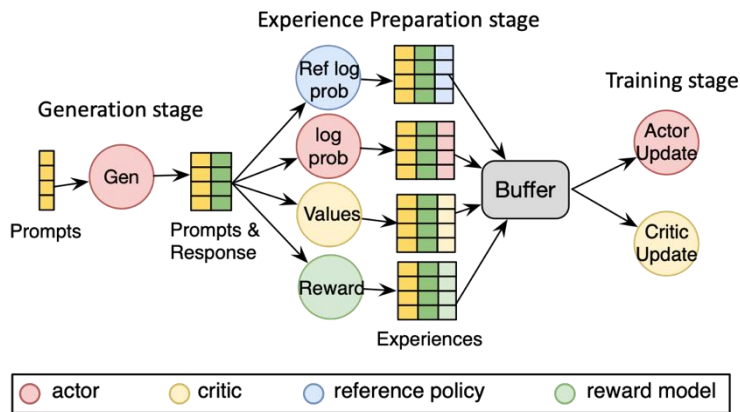


Verl's Distributed Paradigm: Hybrid-Controller



- Hybrid-Controller = Single-Controller + N x Multi-Controller
- In the hybrid-controller, a single-controller manages multiple multi-controllers to process the dataflow

Flexibility in Programming: “Single-Controller”



```
1 for prompts in dataloader:
2     # Stage 1: Generation
3     batch = actor.generate_sequences(prompts)
4     # Stage 2: Experience Preparation
5     batch = reward.compute_reward(batch)
6     batch = reference.compute_log_prob(batch)
7     batch = critic.compute_values(batch)
8     batch = compute_advantage(batch, "gae")
9     # Stage 3: Training
10    critic.update_critic(batch)
11    actor.update_actor(batch)
```

Programming interface based on the “**single-controller**” paradigm

With single-controller, RL algorithm core logic is implemented in **a few lines of code!** Facilitate diverse RL algorithms

like: [PPO](#), [GRPO](#), [RLOO](#), [ReMax](#), [PRIME](#), [DAPO](#), etc.

Efficiency: “Multi-Controller”

“multi-controller” paradigm is efficient for intra-operator executions and includes features like:

Parallelism Algorithms:

- Data Parallelism
- Tensor Parallelism
- Pipeline Parallelism
- Context / Sequence Parallelism
- ...

Efficient Kernels:

- Flash Attention 2/3
- Torch Compile
- Liger Kernel
- ...

Training Backends:

- FSDP
- FSDP2
- Megatron
- torchtitan

Inference Backends:

- vLLM
- SGLang
- ...

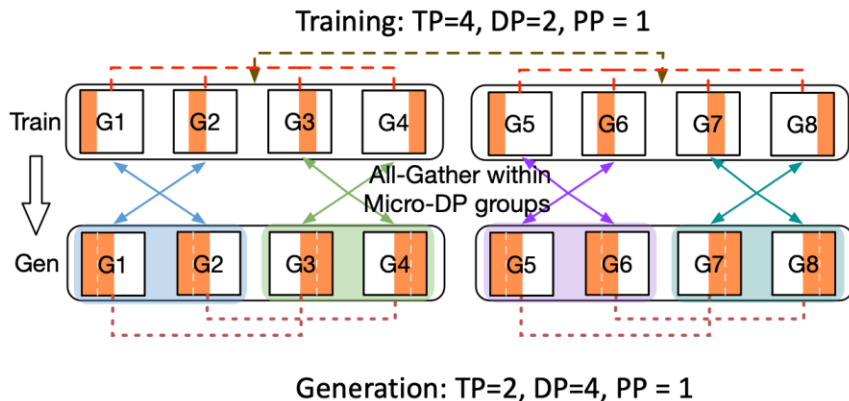
Hybrid Engine: Speedup Collocating Rollout and Trainer

Collocate Strategy: Uses the same grouping strategy in both training and generation stages.

Split Strategy: Uses the different grouping strategy in training and generation stages.

Collocate Strategy is efficient with 3D-HybridEngine:

- Significantly reduces communication overhead during transitions between training and generation phases.
- Offloading & reloading enables fully utilizing the GPU memory



Programming using verl

```
1 for prompts in dataloader:
2     # Stage 1: Generation
3     batch = actor.generate_sequences(prompts)
4     # Stage 2: Experience Preparation
5     batch = reward.compute_reward(batch)
6     batch = reference.compute_log_prob(batch)
7     batch = critic.compute_values(batch)
8     batch = compute_advantage(batch, "gae")
9     # Stage 3: Training
10    critic.update_critic(batch)
11    actor.update_actor(batch)
```

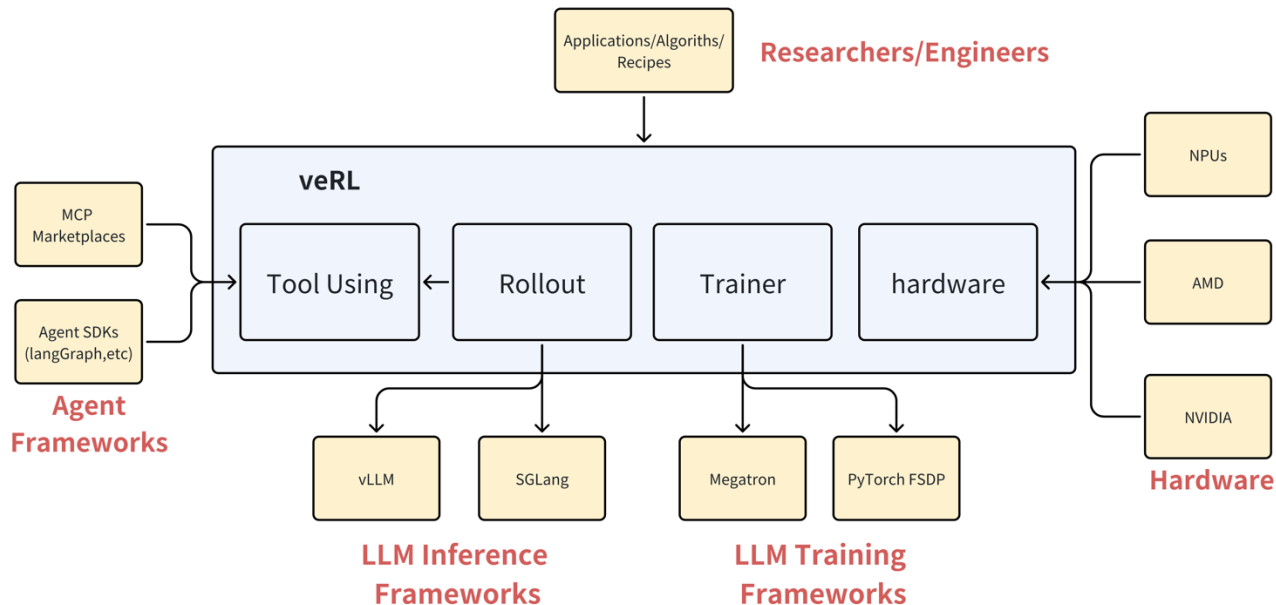
```
1 class CriticWorker(3DParallelWorker):
2     @register(dispatch_mode=3D_PROTO)
3     def compute_values(self, batch: DataProto):
4         values = self.critic.forward(batch)
5         batch.update(values=values)
6     # ...
7 class ActorWorker(3DParallelWorker):
8     @register(dispatch_mode=3D_PROTO)
9     def update_actor(self, batch: DataProto):
10        loss = self.actor(batch)
11        loss.backward()
```

- Each call in the single-controller (e.g. `critic.compute_values`, `actor.update_actor`) is an RPC to a multi-controller worker group
- The register decorator utility manages the distributed data transfer between dataflow nodes, which also makes multi-controller programming easier

Recent Updates & Roadmap

Verl Community

verl is a place connects Researchers and engineers with Agent/LLM Inference/LLM Training/Hardware Communities!



Trainer Updates: Scalable RL for large MoE models

verl has preview versions support RL training for **huge MoE like DeepSeek-V3-671B**, based on the following features:

- Training: **MoE models classes** based on Megatron-Core GPTModel
 - DeepSeek 671B in 96 H20 GPUs ~ 0.15 MFU
 - Qwen3 235B in 32 H20 GPUs ~ 0.2 MFU
- Inference: **Multi-node** inference
- Hybrid: **Parameter sharding manager** for Megatron-Core V0.12 + latest version engines
- **Further Performance Optimization is required!**



For more details, please check:

- <https://verl.readthedocs.io/en/latest/perf/dpsk.html>

Rollout Updates: Multi-turn Rollout with tool using

- Infrastructure for efficient tool utilization
 - asynchronous server-based llm inference engine
 - vLLM/SGLang
- Supported tools includes:
 - Code sandbox (sandbox fusion)
 - Search Engine
 - Computer and browser use
- Apply agent frameworks like LangGraph
- Access to MCP server marketplace
- Easy tracing and debugging for the rollout (wandb Weave)

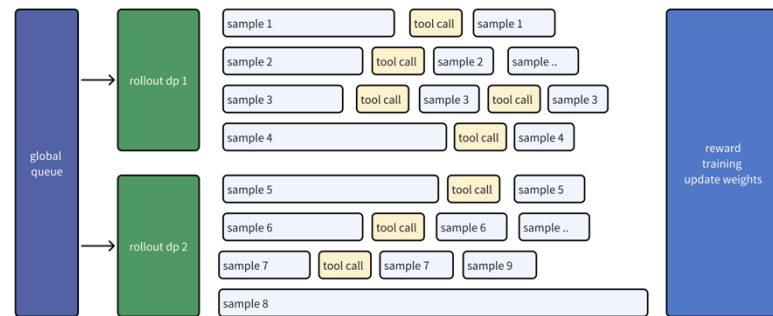
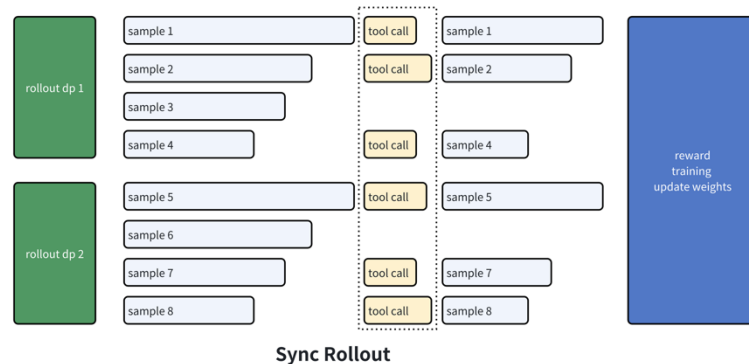


LangGraph

Rollout Updates: Multi-turn Rollout with tool using

Asynchronous server-based llm inference engine (v0.4.0):

1. Request-level multi-turn
2. Overlapping tool-calling with computing
3. Loading balancing Data Parallel with queue pulling



Async Rollout (after verl version 0.4.0)

verl reTool recipe: Using multi-round conversations and code sandboxing to improve the math of large models

Provides end to end retool recipe (*ReTool: Reinforcement Learning for Strategic Tool Use in LLMs*), including:

- Data process
- SFT training script
- Code Sandbox
- RL training script
- Support both vllm and sglang
- Reproducible wandb tracing results

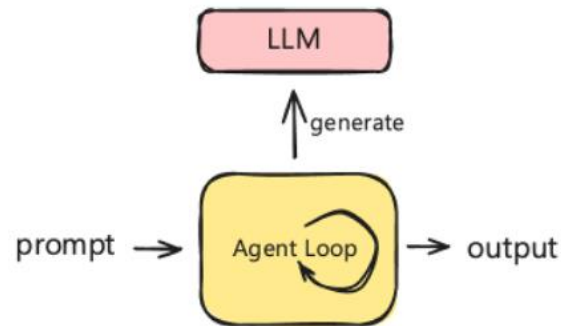
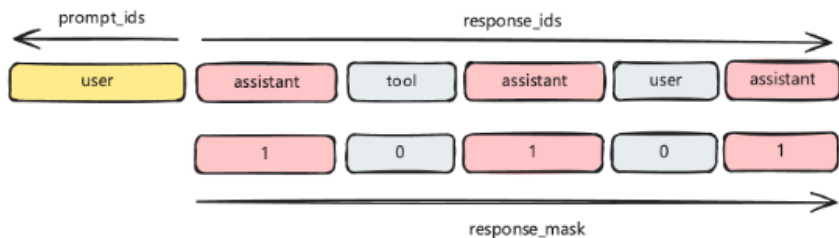
	Turn 1	Turn 2	Turn 3
<reason>	Initial query: Calculate the fractional part of $(2015! / 2017)$, simplify it to min (with m, n coprime), and find $m + n$.	(System automatically verifies) Checks the correctness of the initial result.	(System fixes errors) Resolves the "undefined 'mod' function" issue.
<tool_call>	<pre>from sympy import * from math import factorial import fractions # Calculate 2015! and define prime 2017 factorial_2015 = factorial(2015) p = 2017 # Find remainder of 2015! divided by 2017 remainder = factorial_2015 % p # Convert to simplified fraction result = remainder / p fraction = fractions.Fraction(result).limit_denominator() # Get m, n, and m + n m, n = fraction.numerator, fraction.denominator m, n, m + n</pre>	<pre>from sympy import * # Attempt to calculate remainder with "mod" (incorrect function) remainder = mod(factorial(2015), 2017) # Simplify to fraction and find m + n fraction = Rational(remainder, 2017) m, n = fraction.as_numer_denom() m_plus_n = m + n [m, n, m_plus_n]</pre>	<pre>from sympy import * # Correctly calculate remainder with Python's % operator remainder = factorial(2015) % 2017 # Simplify to fraction and find m + n fraction = Rational(remainder, 2017) m, n = fraction.as_numer_denom() m_plus_n = m + n [m, n, m_plus_n]</pre>
<tool_response>	1 2017 2018	Error: NameError: name 'mod' is not defined	[1, 2017, 2018]

<https://www.notion.so/verl-reTool-recipe-2398b5b7feba80a58156fa936f9f8de6>

Retool : <https://arxiv.org/pdf/2504.11536>

Token-in-token-out Agent Loop Interface

- Given one prompt, run a user defined loop with multi-turn/tool calling trajectories.
- Token ids are used for server generation input / output.

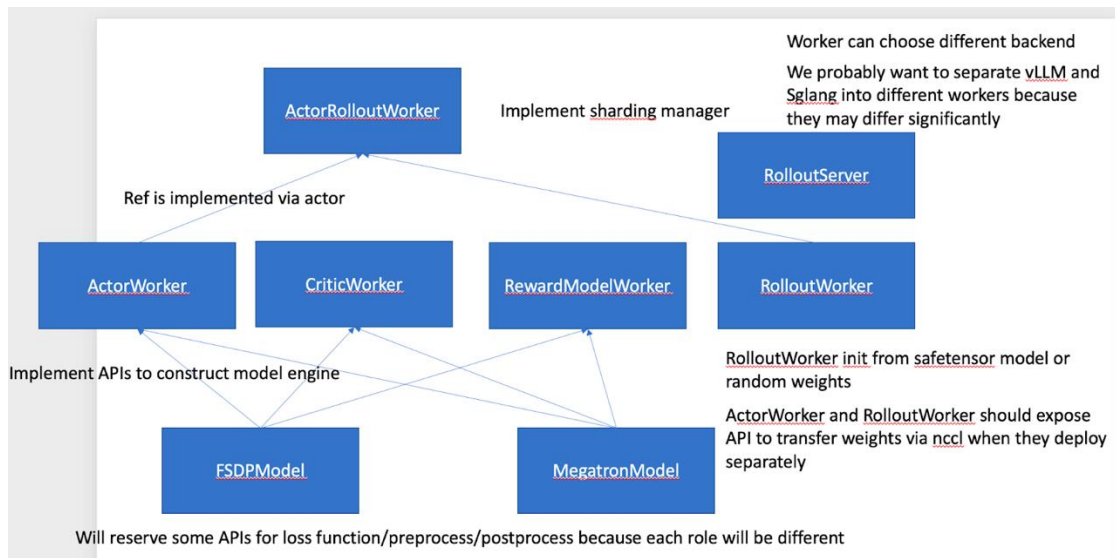


Disaggregated Async Trainer

- Hybrid controller allows flexible device placement (collocate/disaggregated)
- One-step async pipeline to avoid onload/offload overhead & bubbles
- verl/recipe/one_step_off_policy
~1.2x speedup compared to sync trainer
- Long-tail generation problems
- Towards fully-async pipeline
Interruptible generation PR 2200

Q3 Roadmap

- Modular design: composable model engines with better abstraction
 - Algorithm agnostic engine abstraction: FSDP2, Megatron, and more



Q3 Roadmap

- Partial rollout & fully-async training pipeline (AReal, Kimi, 2025)
 - Truncated PPO (Truncated Proximal Policy Optimization)
- Performance optimizations
 - Rollout (fp8)
 - Megatron backend for MoEs
- Agentic RL recipes (e.g. SWE-bench)

Thanks for Listening!

Welcome to join the verl community to [discuss](#) and [contribute](#)!

 Code Repository @ <https://github.com/volcengine/verl>

 Contactor @ haibin.lin@bytedance.com



Wechat Group